

Replication de MNE

Ce code présente une implémentation pratique des analyses de potentiels évoqués (ERP) à l'aide de la bibliothèque MNE-Python. Il utilise les fonctionnalités intégrées de MNE pour traiter les données électrophysiologiques, en s'appuyant sur les modèles et méthodes préétablis dans la librairie. L'approche adoptée permet de réaliser facilement des analyses ERP standards sans avoir à implémenter manuellement les algorithmes de traitement du signal sous-jacents. Le script illustre comment charger les données, extraire les événements à partir d'un fichier externe plutôt que du canal de stimulation, et optimiser les performances en réduisant la durée des données analysées.

Les données importées ont subi un prétraitement initial : les coordonnées 3D des capteurs sont intégrées au fichier .fif, un filtrage passe-bas à 40 Hz a été appliqué, et les canaux EEG utilisent une référence moyenne commune implémentée comme projecteur. Ces étapes seront détaillées plus loin.

Notre jeu de données contenant à la fois des enregistrements EEG et MEG, nous allons d'abord isoler uniquement les canaux EEG et EOG. Cette opération éliminera automatiquement les projecteurs superflus qui concernent exclusivement les magnétomètres. L'analyse des données confirme que notre sélection comprend désormais 59 canaux EEG et 1 canal EOG, prêts pour l'analyse ERP.

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

import mne

root = mne.datasets.sample.data_path() / "MEG" / "sample"
raw_file = root / "sample_audvis_filt-0-40_raw.fif"
raw = mne.io.read_raw_fif(raw_file, preload=False)

events_file = root / "sample_audvis_filt-0-40_raw-eve.fif"
events = mne.read_events(events_file)

raw.crop(tmax=90) # in seconds (happens in-place)
# discard events >90 seconds (not strictly necessary, but avoids some
warnings)
events = events[events[:, 0] <= raw.last_samp]

Opening raw data file
/Users/macbook/mne_data/MNE-sample-data/MEG/sample/sample_audvis_filt-
0-40_raw.fif...
Read a total of 4 projection items:
    PCA-v1 (1 x 102)  idle
    PCA-v2 (1 x 102)  idle
    PCA-v3 (1 x 102)  idle
    Average EEG reference (1 x 60)  idle
```

```
Range : 6450 ... 48149 = 42.956 ... 320.665 secs
Ready.
```

Ici on peut trouver les info dans notre fichier .fif.

On peut voir que les données EEG ont été enregistrées le 3 décembre 2002 à 19h01 avec une fréquence d'échantillonnage de 150,15 Hz. L'enregistrement comprend 59 canaux EEG (dont un défectueux) et un canal EOG, avec une digitalisation de 146 points pour la tête et les capteurs. Des filtres passe-haut à 0,10 Hz et passe-bas à 40,00 Hz ont été appliqués. Une référence moyenne EEG a été calculée mais n'est pas activée. L'identité du participant et de l'expérimentateur est inconnue.

```
raw.pick(["eeg", "eog"]).load_data()
raw.info

Reading 0 ... 13514 = 0.000 ... 90.001 secs...

<Info | 14 non-empty values
bads: 1 items (EEG 053)
ch_names: EEG 001, EEG 002, EEG 003, EEG 004, EEG 005, EEG 006, EEG
007, ...
chs: 60 EEG, 1 EOG
custom_ref_applied: False
dev_head_t: MEG device -> head transform
dig: 146 items (3 Cardinal, 4 HPI, 61 EEG, 78 Extra)
highpass: 0.1 Hz
hpi_meas: 1 item (list)
hpi_results: 1 item (list)
lowpass: 40.0 Hz
meas_date: 2002-12-03 19:01:10 UTC
meas_id: 4 items (dict)
nchan: 61
projs: Average EEG reference: off
sfreq: 150.2 Hz
>
```

Noms et types de electrodes

Il n'est pas rare que certains electrodes soient incorrectement étiquetés, par exemple des canaux EOG identifiés comme EEG. Pour corriger cela, les objets Raw proposent la méthode `set_channel_types()` qui permet de modifier le type d'un electrode mal classé (par exemple, passer de eeg à eog).

Il est également possible de renommer les electrodes à l'aide de la méthode `rename_channels()`.

Dans notre jeu de données, les types de electrodes sont déjà corrects. Nous allons donc simplement nettoyer les noms en supprimant les espaces et les zéros initiaux, puis en les convertissant en minuscules.

```
channel_renaming_dict = {name: name.replace(" 0", "").lower() for name
in raw.ch_names}
_ = raw.rename_channels(channel_renaming_dict) # happens in-place
```

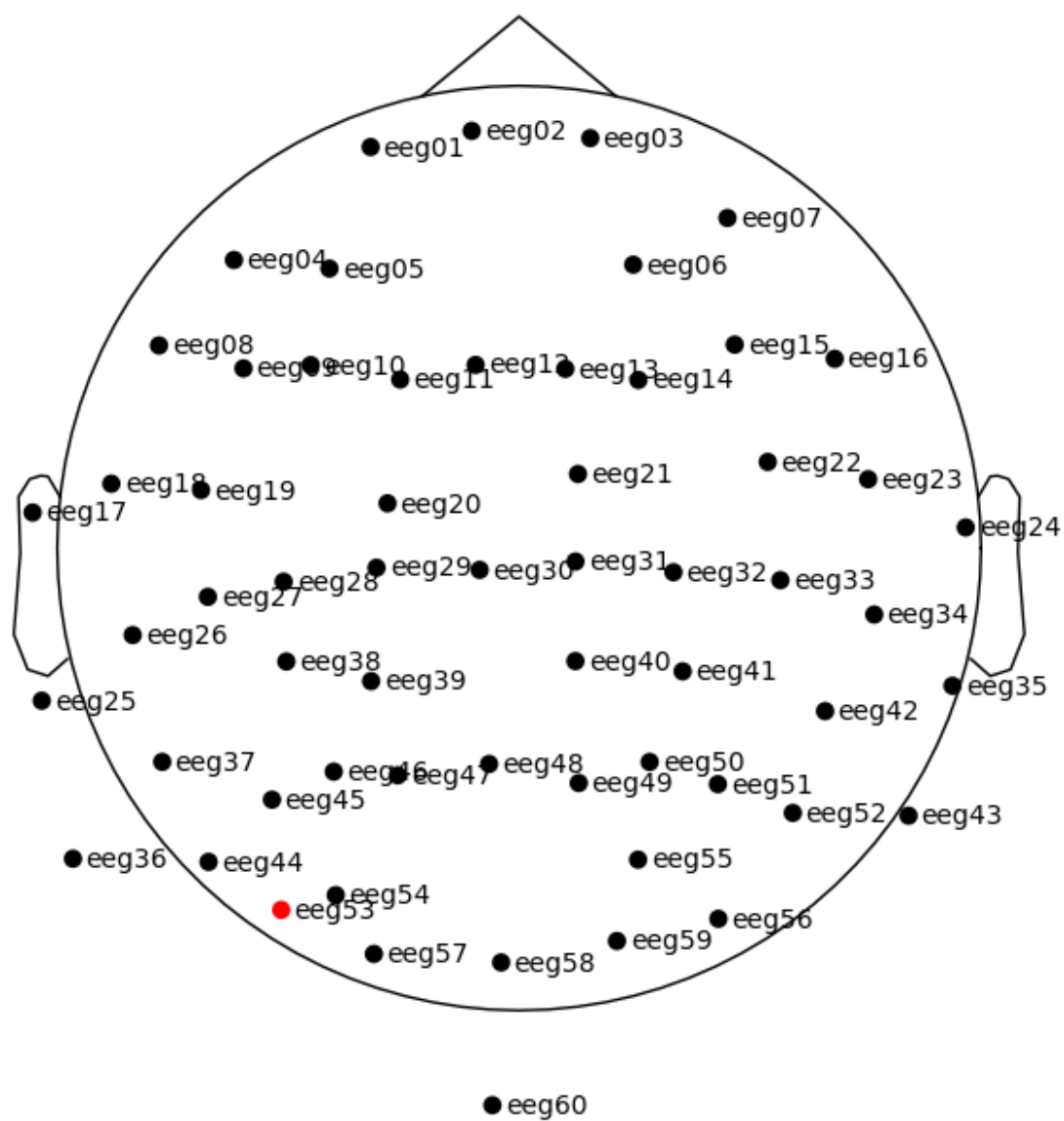
Emplacement des electrodes

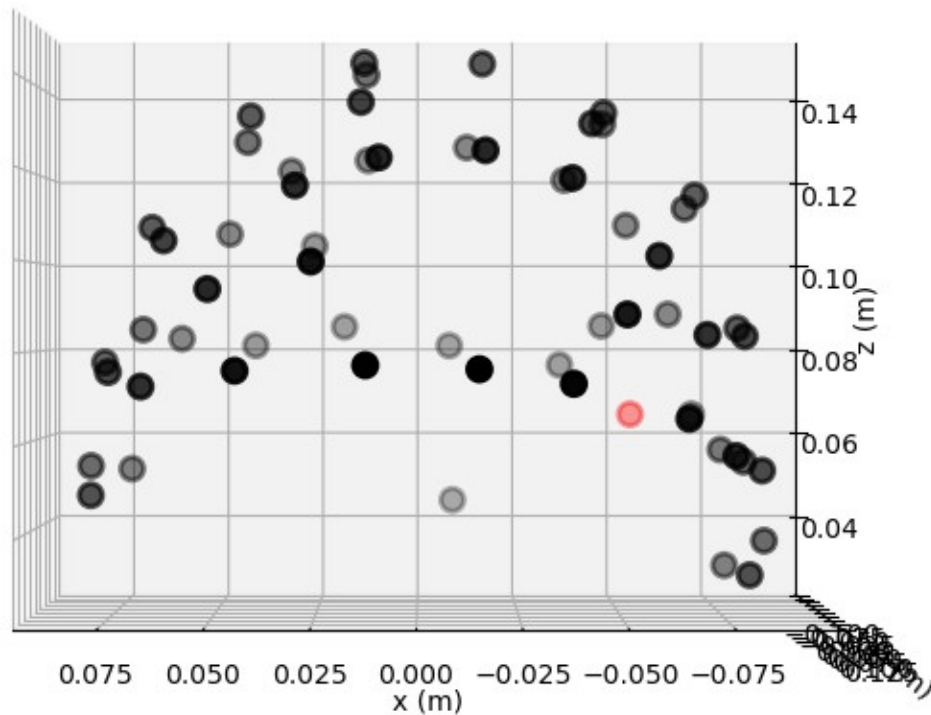
MNE-Python distingue deux concepts : les montages, qui fournissent les coordonnées 3D des capteurs (x, y, z, en mètres), et les layouts, qui définissent l'agencement 2D des electrodes pour l'affichage des schémas de localisation.

Les montages peuvent contenir soit des positions idéalisées des electrodes (par exemple, basées sur un modèle de tête sphérique), soit des positions réalistes obtenues par numérisation des capteurs placés sur la tête d'un sujet.

Dans notre cas, le jeu de données contient des positions 3D réalistes, enregistrées dans le fichier .fif. Il est donc possible de visualiser la disposition des electrodes en 2D ou en 3D à l'aide de la méthode `plot_sensors()`.

```
raw.plot_sensors(show_names=True)
fig = raw.plot_sensors("3d")
```





Si vous travaillez avec un montage standard tel que le système 10-20, vous pouvez ajouter les emplacements des electrodes à vos données à l'aide de `raw.set_montage('standard_1020')`.

Si vous disposez d'emplacements de electrode réalistes obtenus par numérisation, MNE-Python propose des fonctions spécifiques pour charger ces fichiers.

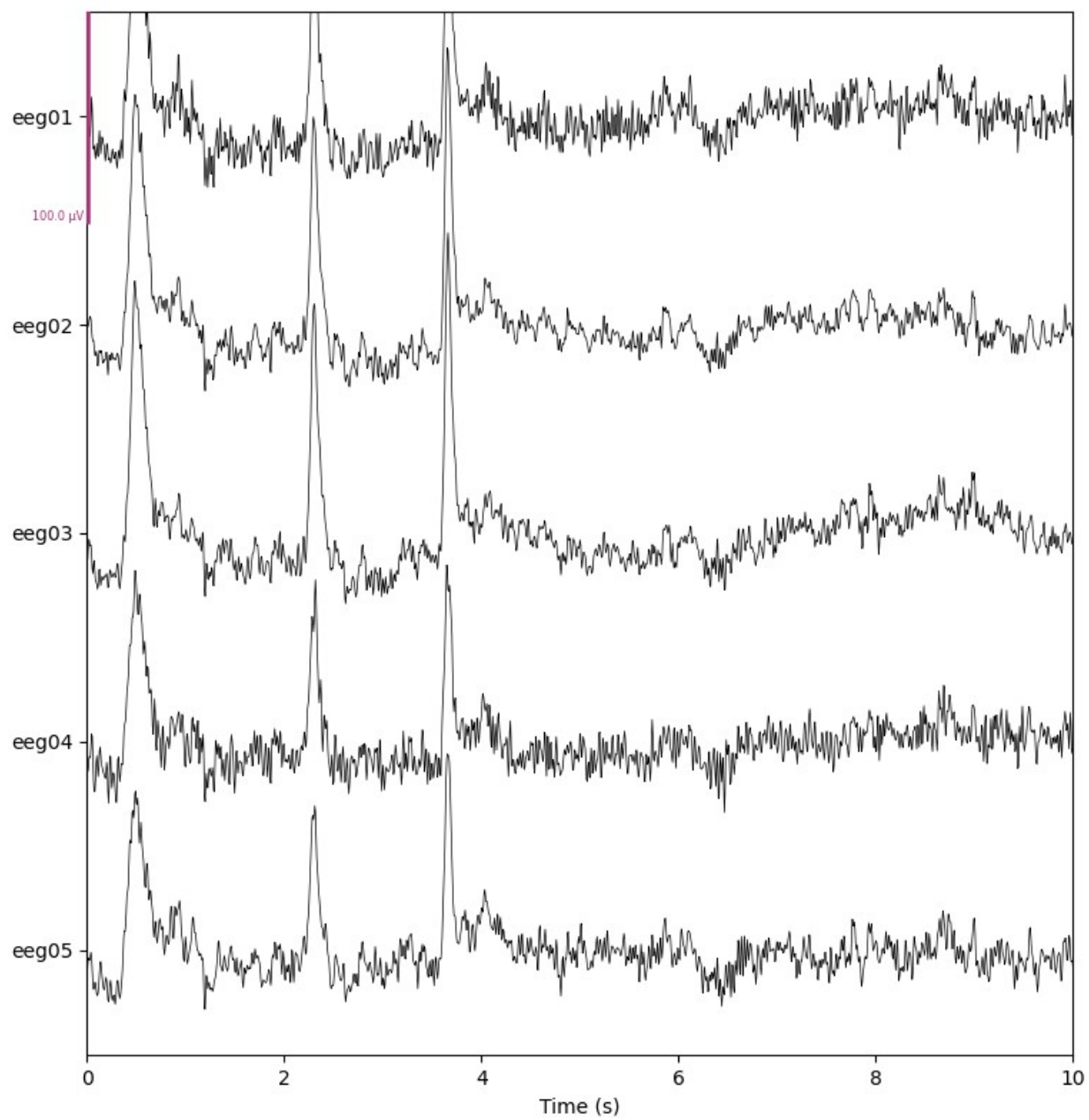
Une fois les données de numérisation chargées, il suffit de les transmettre à `set_montage()` pour les associer aux données EEG.

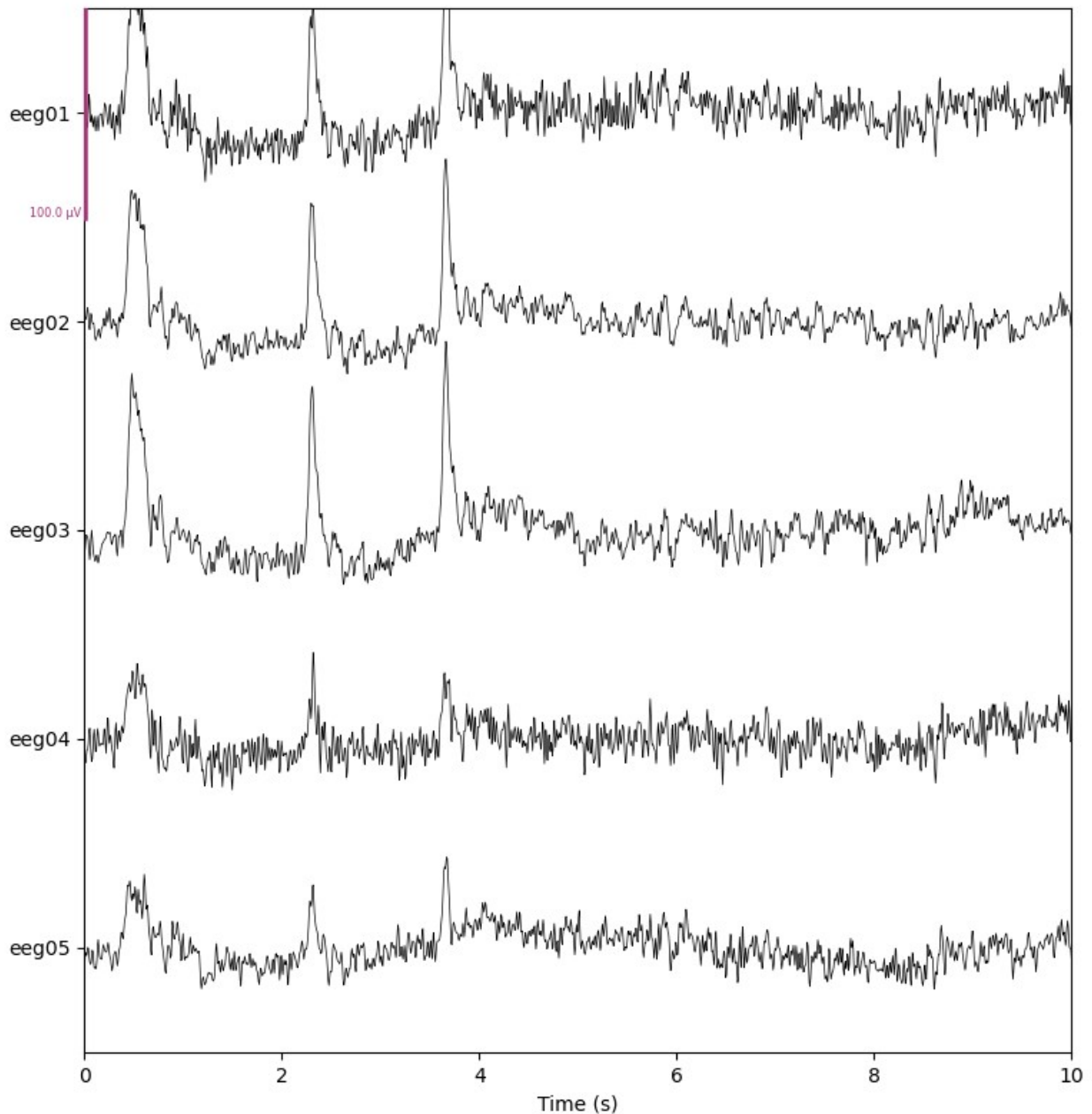
Configuration de la référence EEG

Ces données incluent déjà une référence moyenne commune EEG, ajoutée sous forme de projecteur. Nous pouvons visualiser l'effet de ce projecteur sur les données brutes en comparant des représentations graphiques avec et sans son application.

```
for proj in (False, True):  
    with mne.viz.use_browser_backend("matplotlib"):  
        fig = raw.plot(  
            n_channels=5, proj=proj, scalings=dict(eeg=50e-6),  
            show_scrollbars=False  
        )  
        fig.subplots_adjust(top=0.9) # make room for title  
        ref = "Average" if proj else "No"  
        fig.suptitle(f"{ref} reference", size="xx-large", weight="bold")
```

Using matplotlib as 2D backend.





La méthode de référencement peut être modifiée à l'aide de la fonction `mne.set_eeg_reference()` (qui agit par défaut sur une copie des données), ou bien avec la méthode `raw.set_eeg_reference()` (qui, elle, modifie directement les données originales).

Filtrage

MNE-Python propose une prise en charge étendue de différentes méthodes de filtrage des données.

Ici, nous allons appliquer un simple filtre passe-haut à titre d'illustration :

```
raw.filter(l_freq=0.1, h_freq=None)

Filtering raw data in 1 contiguous segment
Setting up high-pass filter at 0.1 Hz

FIR filter parameters
-----
Designing a one-pass, zero-phase, non-causal highpass filter:
- Windowed time-domain design (firwin) method
- Hamming window with 0.0194 passband ripple and 53 dB stopband
attenuation
- Lower passband edge: 0.10
- Lower transition bandwidth: 0.10 Hz (-6 dB cutoff frequency: 0.05
Hz)
- Filter length: 4957 samples (33.013 s)

[Parallel(n_jobs=1)]: Done 17 tasks      | elapsed:    0.0s
<Raw | sample_audvis_filt-0-40_raw.fif, 61 x 13515 (90.0 s), ~9.2 MiB,
data loaded>
```

Réponses évoquées : segmentation en epochs et moyennage

Pour extraire des réponses évoquées à partir de données EEG continues, on commence généralement par créer des epochs à l'aide du constructeur Epochs, puis on calcule la moyenne de ces epochs pour obtenir un objet Evoked.

Dans MNE-Python, les événements sont représentés par un tableau NumPy qui contient les latences (exprimées en nombre d'échantillons) ainsi que des codes d'événements entiers. Ces codes sont situés dans la dernière colonne du tableau events.

```
np.unique(events[:, -1])

array([ 1,  2,  3,  4,  5, 32])
```

Dans MNE-Python, les codes d'événements (des entiers) peuvent être associés à des noms plus explicites grâce à un dictionnaire Python, généralement nommé event_id.

Cette correspondance dépend de votre protocole expérimental : elle reflète les choix de codage que vous avez faits pour représenter les différents types d'événements ou conditions.

Le jeu de données Sample, par exemple, utilise la correspondance suivante :

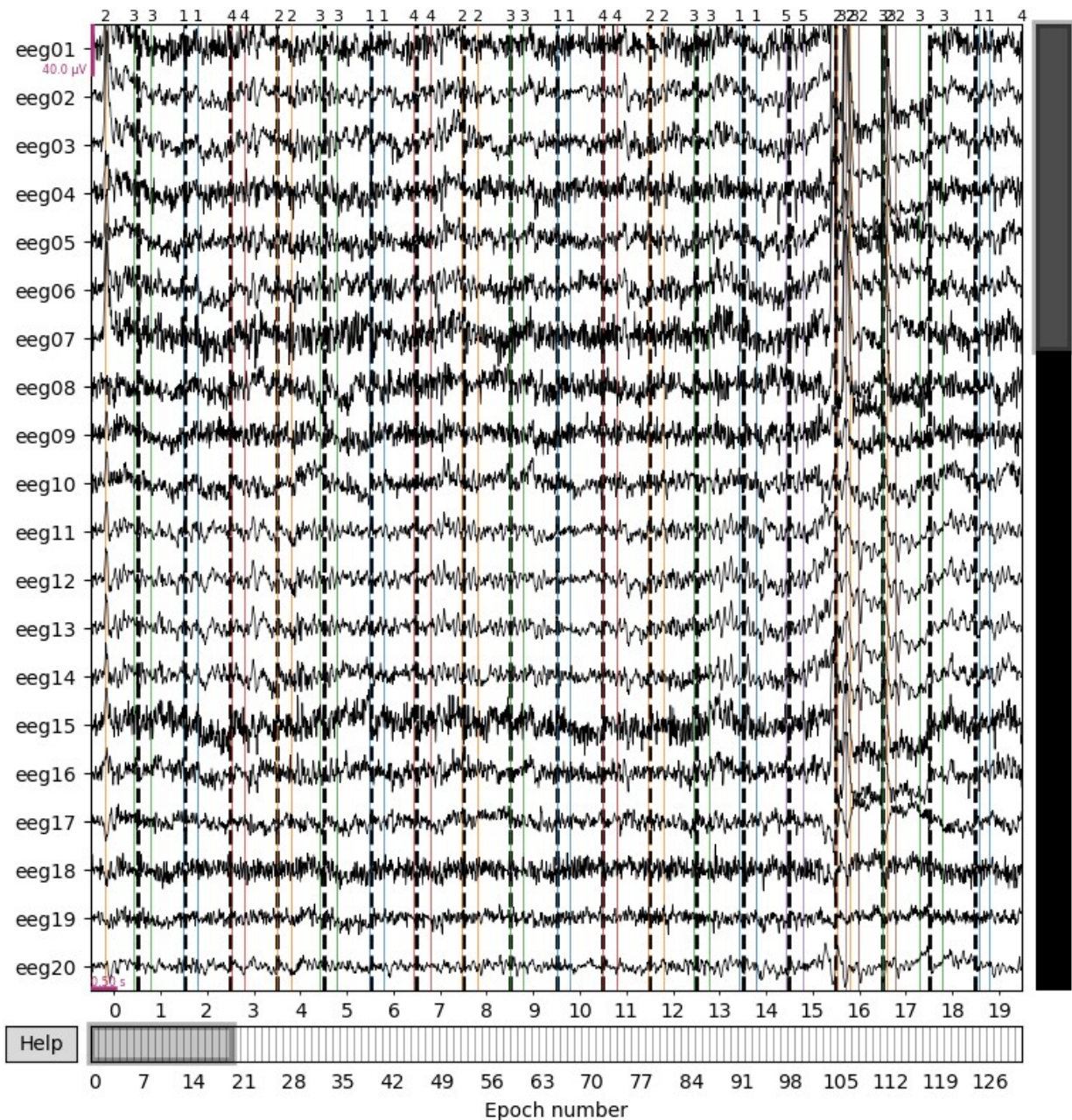
```
event_dict = {
    "auditory/left": 1,
```

```
"auditory/right": 2,  
"visual/left": 3,  
"visual/right": 4,  
"face": 5,  
"buttonpress": 32,  
}
```

Nous pouvons maintenant segmenter les données continues en epochs. Un graphique interactif permet de cliquer sur les epochs pour les marquer comme "mauvaises" et les exclure de l'analyse (bien que cette fonctionnalité ne soit pas interactive sur ce site de documentation, elle le sera lorsque vous exécuterez `epochs.plot()` dans une console Python).

```
epochs = mne.Epochs(raw, events, event_id=event_dict, tmin=-0.3,  
tmax=0.7, preload=True)  
fig = epochs.plot(events=events)
```

```
Not setting metadata  
132 matching events found  
Setting baseline interval to [-0.2996928197375818, 0.0] s  
Applying baseline correction (mode: mean)  
Created an SSP operator (subspace dimension = 1)  
1 projection items activated  
Using data from preloaded Raw for 132 events and 151 original time  
points ...  
1 bad epochs dropped  
You seem to have overlapping epochs. Some event lines may be  
duplicated in the plot.
```



Il est également possible de supprimer automatiquement des epochs, soit lors de leur création, soit ultérieurement, en définissant des seuils d'amplitude maximale du signal crête-à-crête (ce paramètre est passé à Epochs sous le nom reject ; pour plus de détails, consultez Rejeter les epochs en fonction de l'amplitude crête-à-crête des canaux).

Vous pouvez aussi procéder à cette suppression après la création des epochs en utilisant la méthode `drop_bad()`.

```
reject_criteria = dict(eeg=100e-6, eog=200e-6) # 100 μV, 200 μV
epochs.drop_bad(reject=reject_criteria)
```

```

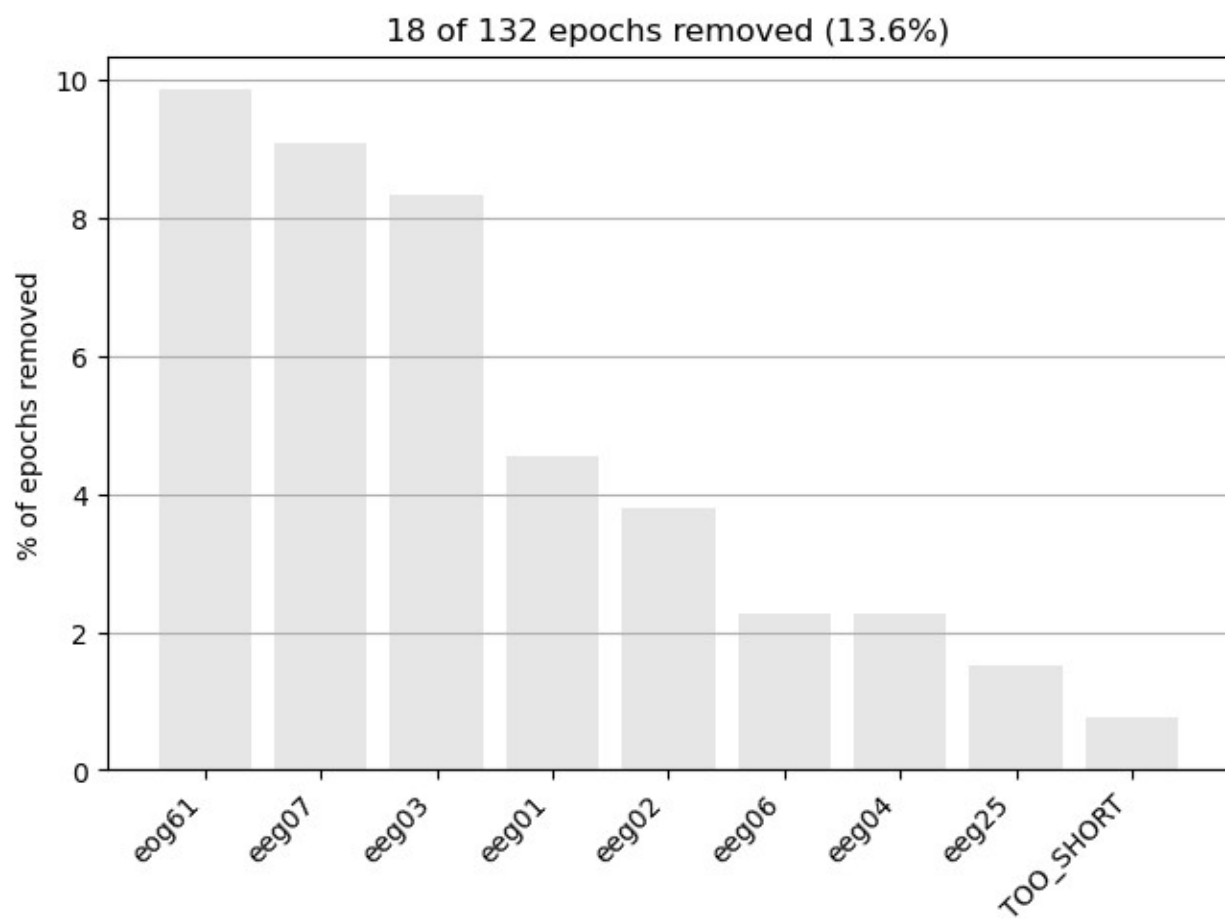
    Rejecting epoch based on EEG : ['eeg03']
    Rejecting epoch based on EEG : ['eeg01', 'eeg02', 'eeg03',
'eeg04', 'eeg06', 'eeg07']
    Rejecting epoch based on EEG : ['eeg01', 'eeg02', 'eeg03',
'eeg04', 'eeg06', 'eeg07']
    Rejecting epoch based on EEG : ['eeg01', 'eeg02', 'eeg03',
'eeg04', 'eeg06', 'eeg07']
    Rejecting epoch based on EEG : ['eeg01', 'eeg02', 'eeg03',
'eeg07']
    Rejecting epoch based on EEG : ['eeg01', 'eeg02', 'eeg03',
'eeg07']
    Rejecting epoch based on EEG : ['eeg01']
    Rejecting epoch based on EEG : ['eeg03', 'eeg07']
    Rejecting epoch based on EEG : ['eeg03', 'eeg07']
    Rejecting epoch based on EEG : ['eeg07']
    Rejecting epoch based on EEG : ['eeg03', 'eeg07']
    Rejecting epoch based on EEG : ['eeg03', 'eeg07']
    Rejecting epoch based on EEG : ['eeg07']
    Rejecting epoch based on EEG : ['eeg07']
    Rejecting epoch based on EEG : ['eeg03']
    Rejecting epoch based on EEG : ['eeg25']
    Rejecting epoch based on EEG : ['eeg25']
17 bad epochs dropped

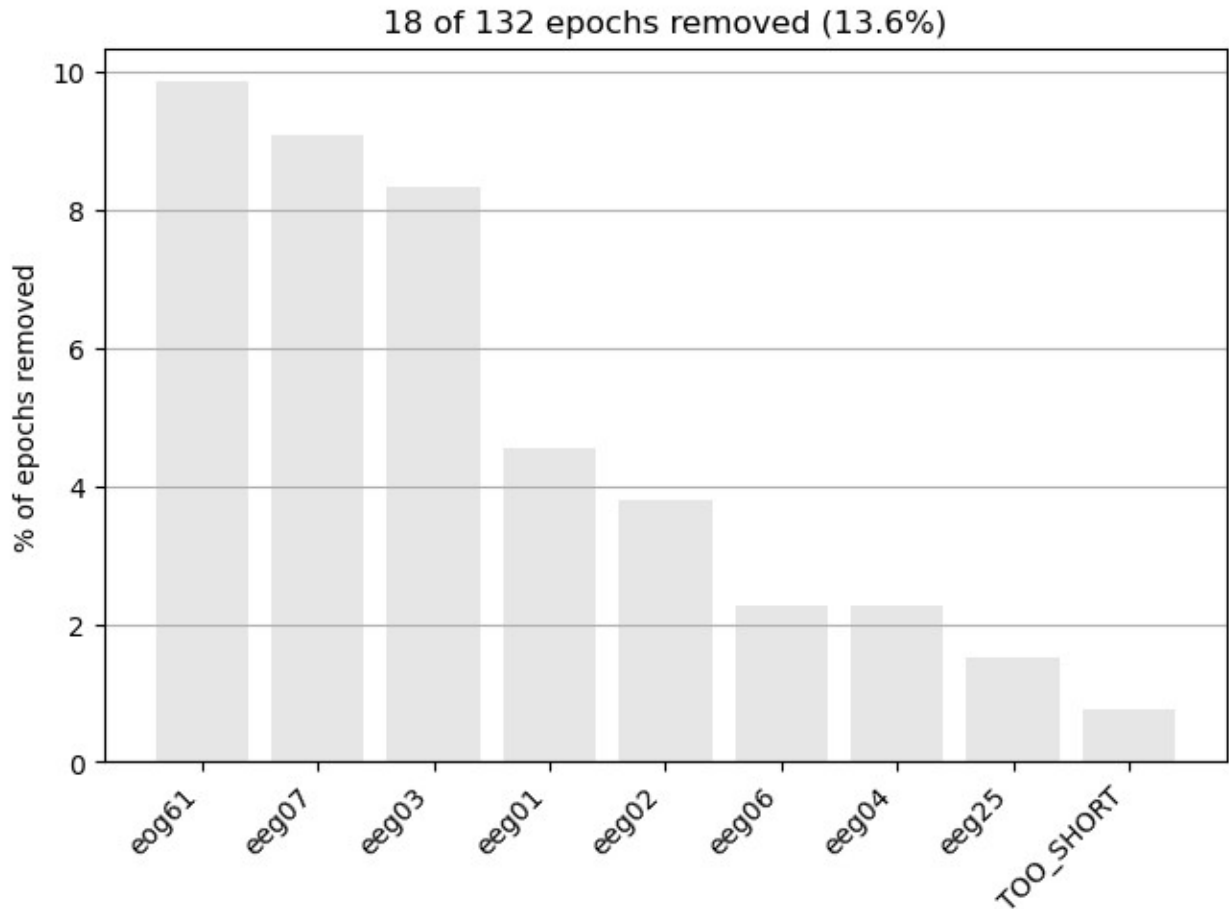
<Epochs | 114 events (all good), -0.3 – 0.699 s (baseline -0.3 – 0 s),
~10.9 MiB, data loaded,
'auditory/left': 25
'auditory/right': 27
'visual/left': 28
'visual/right': 25
'face': 5
'buttonpress': 4>

```

Ensuite, nous générons un graphique en barres pour identifier quels canaux ont le plus contribué aux rejets d'epochs. Si un electrode est responsable de nombreux rejets, il pourrait être utile de le marquer comme "mauvais" dans l'objet Raw et de relancer la segmentation. En effet, il peut être préférable d'avoir moins de canaux avec davantage d'epochs valides plutôt que de conserver tous les canaux tout en perdant de nombreuses epochs.

```
epochs.plot_drop_log()
```





Les epochs peuvent également être supprimées automatiquement si l'événement autour duquel l'epoch est créé est trop proche du début ou de la fin de l'objet Raw (par exemple, si l'epoch dépasse la fin de l'enregistrement, ce qui est à l'origine de l'entrée "TOO_SHORT" dans le graphique généré par `plot_drop_log()`).

De plus, les epochs peuvent être rejetées automatiquement si l'objet Raw contient des annotations commençant par "bad" ou "edge" (les annotations "edge" sont ajoutées automatiquement lors de la concaténation de plusieurs objets Raw).

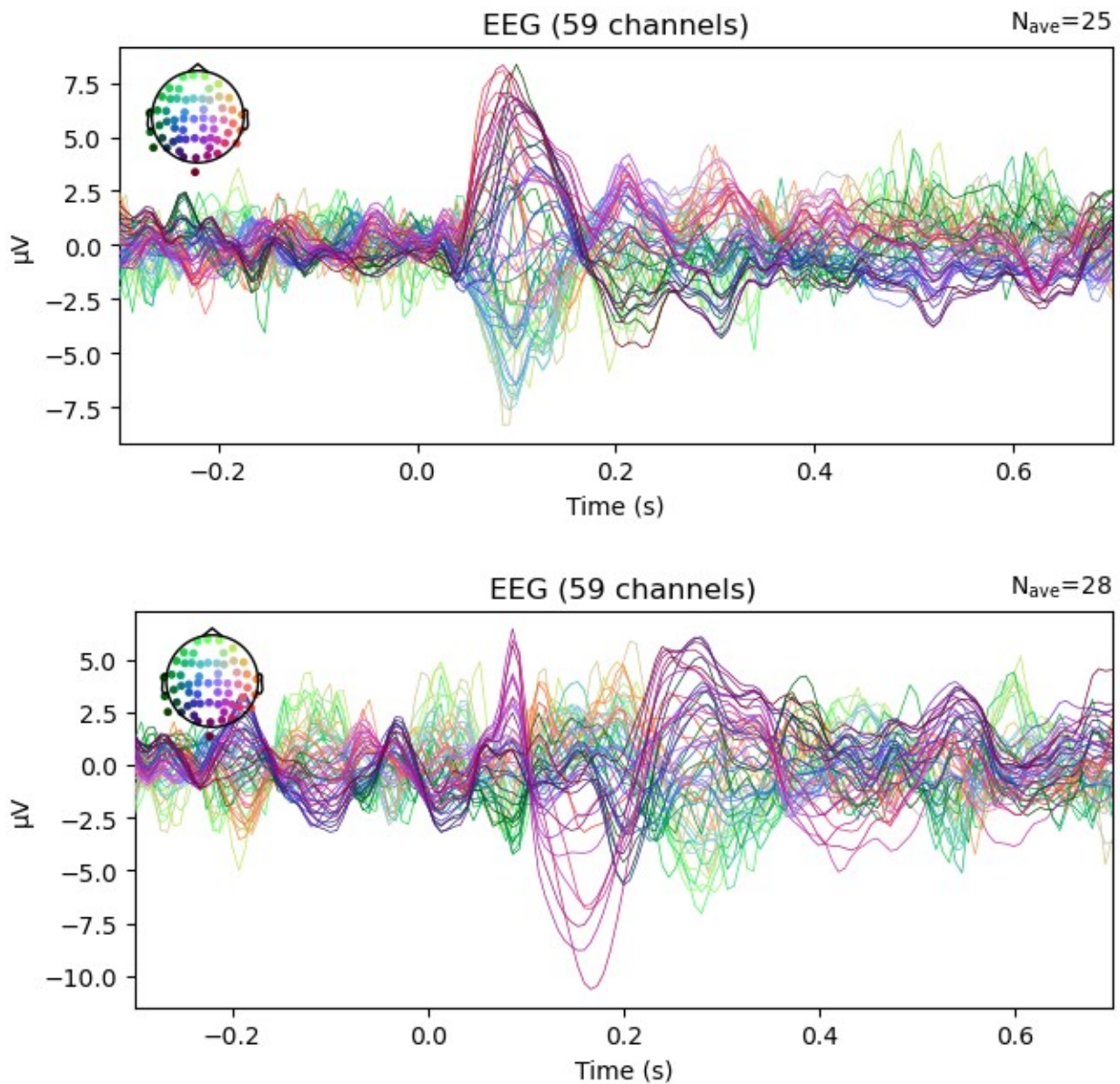
Maintenant que nous avons éliminé toutes les mauvaises epochs, examinons les réponses évoquées pour les conditions qui nous intéressent. La méthode `average()` va créer un objet `Evoked`, que nous pourrions ensuite visualiser. Notez que nous sélectionnons la condition à moyenniser à l'aide de l'indexation par crochets (comme pour un dictionnaire). Cela nous donne un sous-ensemble avec uniquement les epochs sélectionnées, que nous pourrions ensuite moyenniser.

```
l_aud = epochs["auditory/left"].average()  
l_vis = epochs["visual/left"].average()
```

Les objets `Evoked` disposent de leur propre méthode de tracé interactive (bien que cette fonctionnalité ne soit pas interactive sur le site de documentation). En cliquant et en faisant glisser un intervalle de temps, on peut générer une topographie des potentiels du cuir chevelu

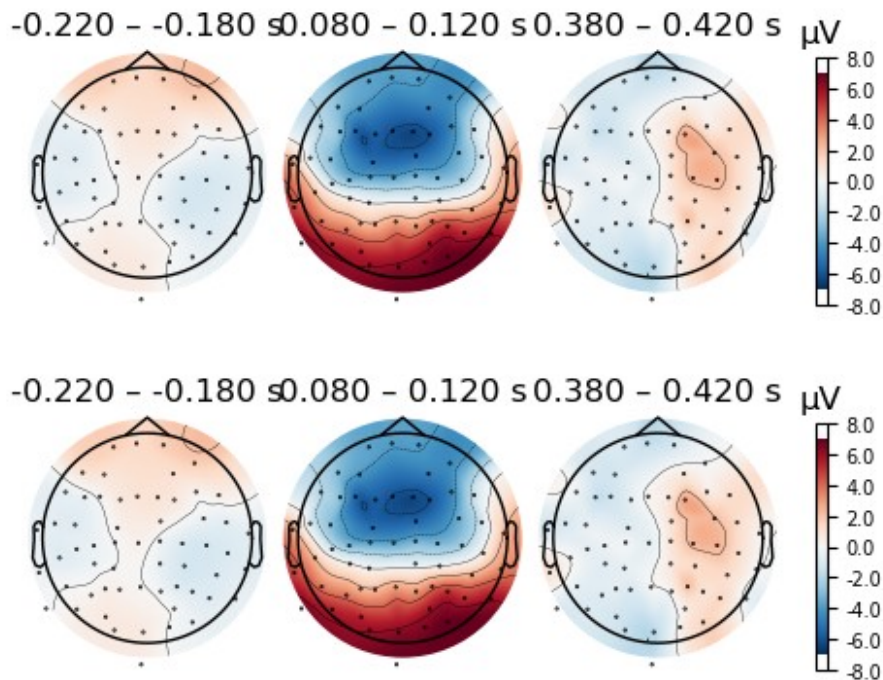
pour la période sélectionnée. Dans cet exemple, nous illustrons également le codage couleur intégré qui permet de visualiser les traces des canaux en fonction de leur emplacement.

```
fig1 = l_aud.plot()  
fig2 = l_vis.plot(spatial_colors=True)
```



Les topographies du cuir chevelu peuvent aussi être générées de manière non interactive en utilisant la méthode `plot_topomap()`. Dans cet exemple, nous affichons des topomaps du potentiel évoqué moyen dans des fenêtres de 50 ms centrées sur -200 ms, 100 ms et 400 ms.

```
l_aud.plot_topomap(times=[-0.2, 0.1, 0.4], average=0.05)
```

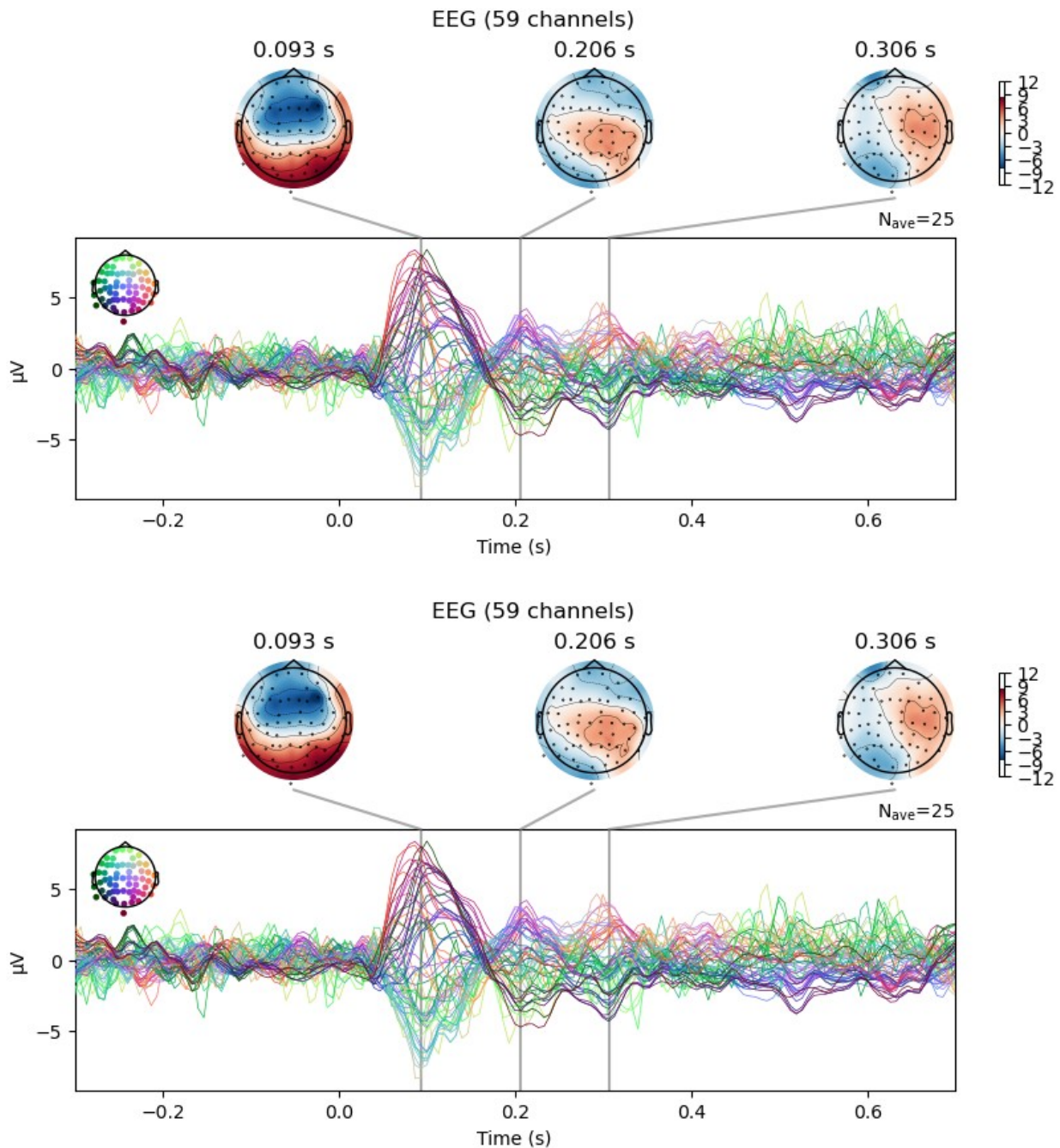


Ces graphiques peuvent être largement personnalisés, pour plus de détails, consultez la docstring de `plot_topomap()`.

Une autre méthode intégrée permet de combiner des graphiques en butterfly des signaux avec des topographies du cuir chevelu, il s'agit de `plot_joint()`. Comme pour `plot_topomap()`, vous pouvez spécifier des moments pour les topographies du cuir chevelu ou laisser la méthode choisir les moments automatiquement, comme le montre l'exemple ci-dessous :

```
l_aud.plot_joint()
```

```
Projections have already been applied. Setting proj attribute to True.
```



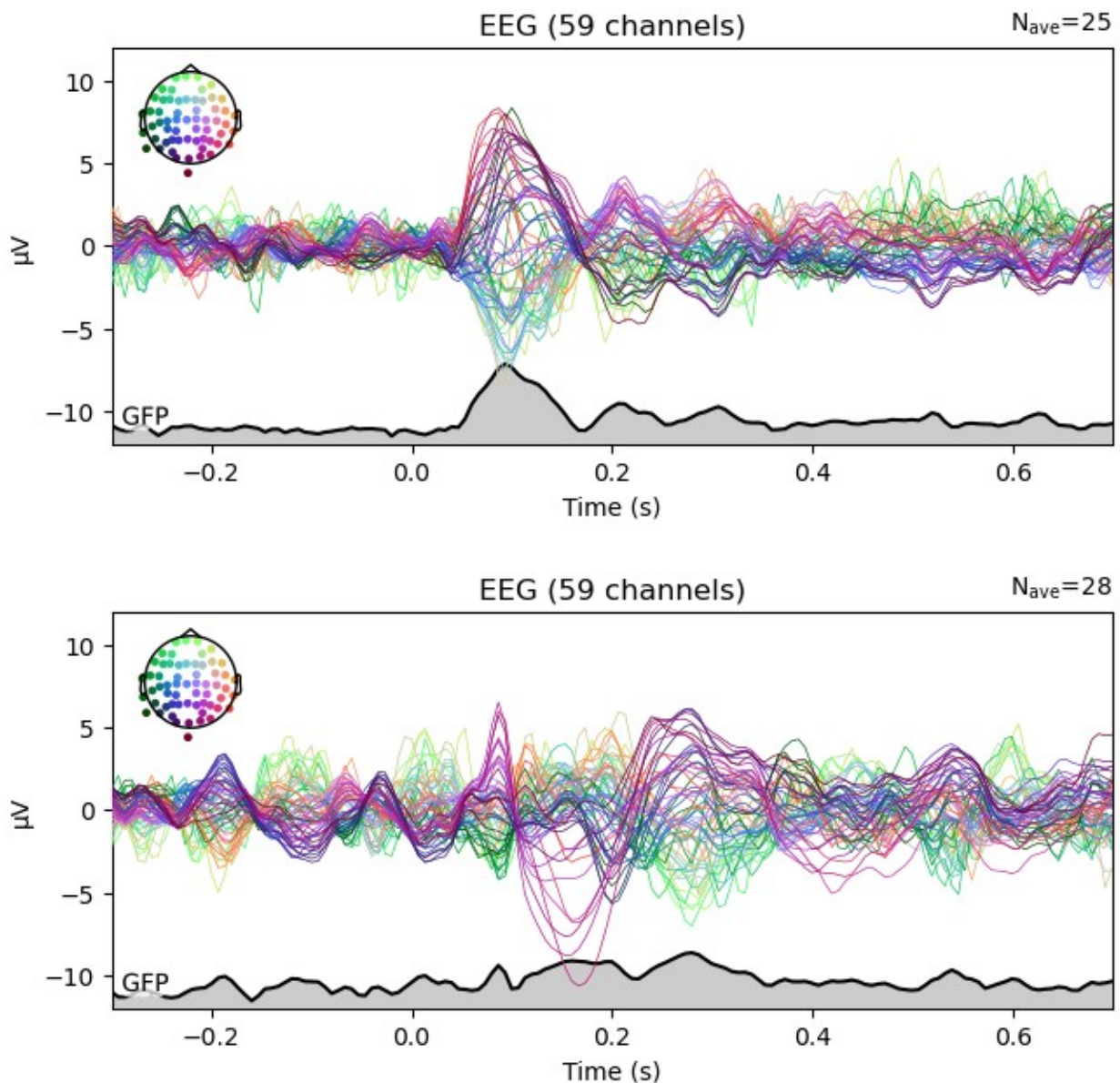
Puissance du champ global (GFP)

La puissance du champ global (GFP) est, de manière générale, une mesure de la cohérence des signaux captés par tous les électrodes sur l'ensemble du cuir chevelu : si tous les électrodes enregistrent la même valeur à un instant donné, la GFP sera nulle à ce moment-là. En revanche, si les signaux diffèrent, la GFP sera non nulle. Les pics de GFP peuvent indiquer une activité cérébrale "significative", justifiant un examen plus approfondi. Mathématiquement, la GFP

correspond à l'écart-type global des signaux de tous les capteurs, calculé séparément pour chaque instant.

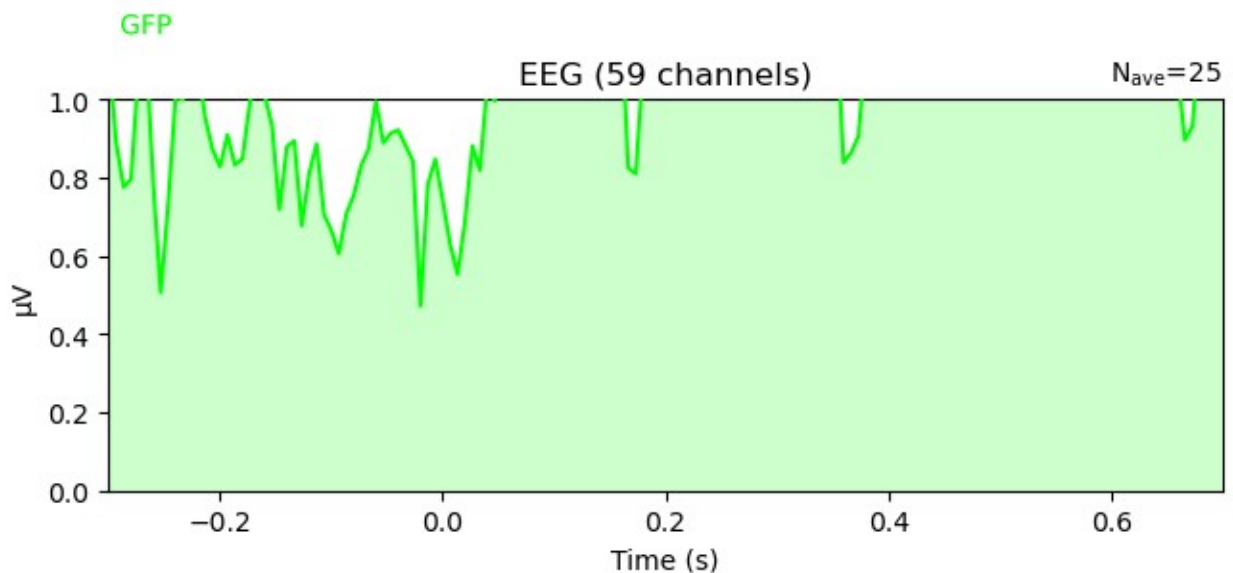
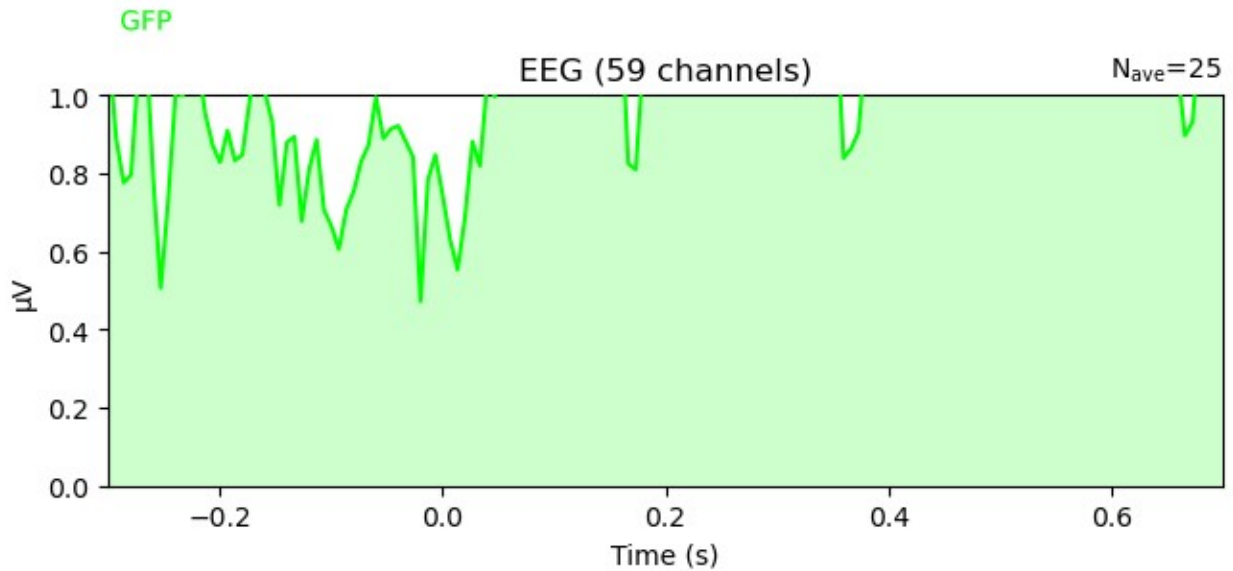
Vous pouvez visualiser la GFP en utilisant `evoked.plot(gfp=True)`. La trace de la GFP sera noire si `spatial_colors=True` et verte dans le cas contraire. La référence EEG n'a pas d'impact sur la GFP :

```
for evk in (l_aud, l_vis):  
    evk.plot(gfp=True, spatial_colors=True, ylim=dict(eeg=[-12, 12]))
```



Pour tracer uniquement le GFP, vous pouvez passer `gfp='only'` (cela facilite la lecture des valeurs des données GFP, car l'échelle est alignée).

```
l_aud.plot(gfp="only")
```

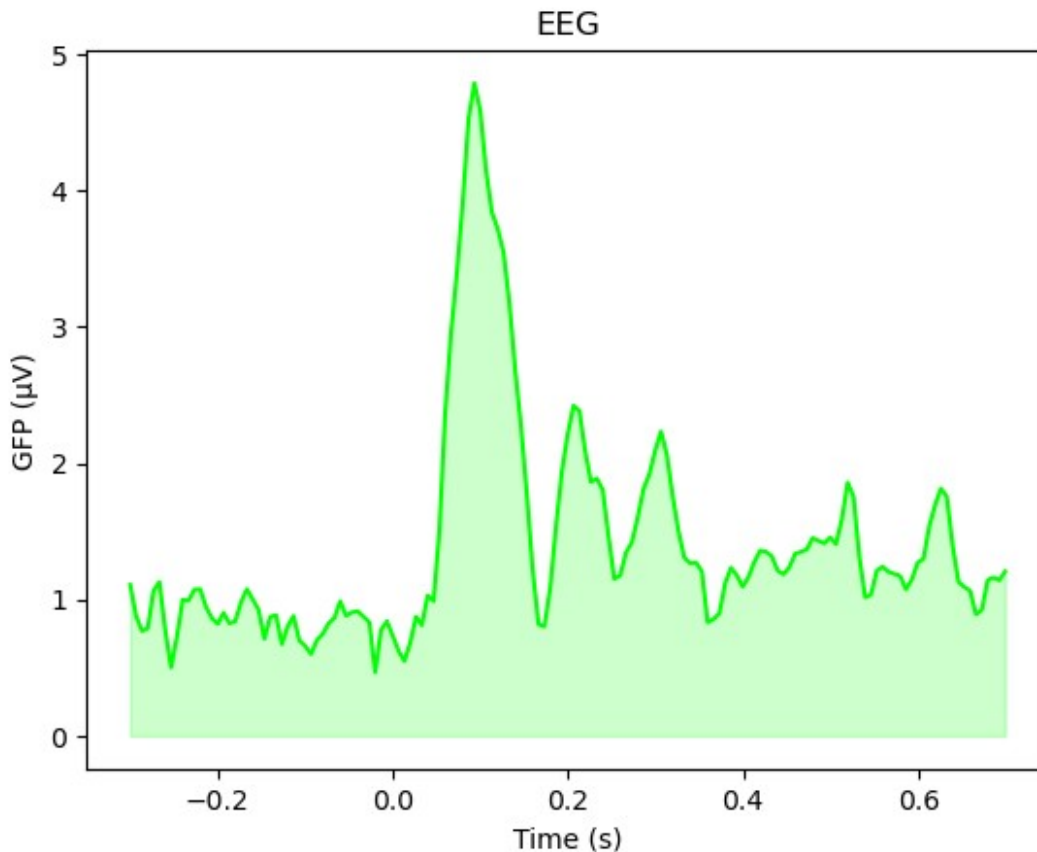


Le GFP est l'écart-type de la population du signal à travers les canaux. Pour le calculer manuellement, nous pouvons tirer parti du fait que `evoked.data` est un tableau NumPy, et le vérifier en le traçant à l'aide de commandes Matplotlib simples.

```
gfp = l_aud.data.std(axis=0, ddof=0)

# Reproducing the MNE-Python plot style seen above
fig, ax = plt.subplots()
ax.plot(l_aud.times, gfp * 1e6, color="lime")
ax.fill_between(l_aud.times, gfp * 1e6, color="lime", alpha=0.2)
ax.set(xlabel="Time (s)", ylabel="GFP ( $\mu V$ )", title="EEG")
```

```
[Text(0.5, 0, 'Time (s)'), Text(0, 0.5, 'GFP ( $\mu$ V)'), Text(0.5, 1.0, 'EEG')]
```



Moyenne des canaux par régions d'intérêt

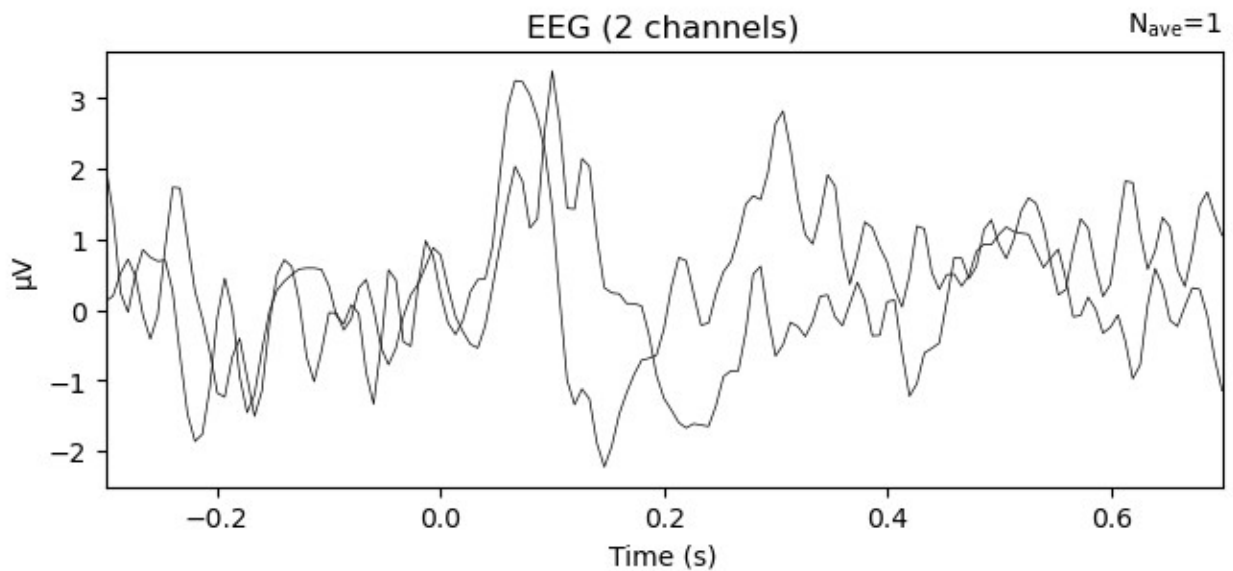
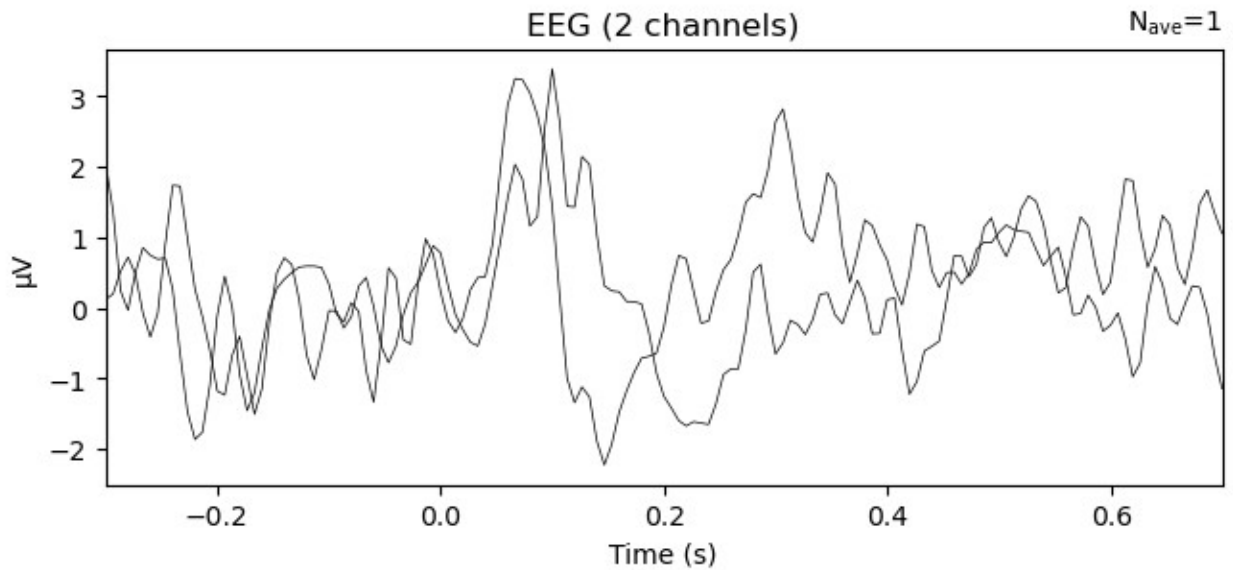
Afin d'obtenir une moyenne sur les canaux pour des régions d'intérêt (ROIs) spécifiques, étant donné que nos données d'exemple contiennent des réponses à des stimuli auditifs et visuels gauches et droits, nous pourrions vouloir comparer les ROIs gauches et droits. Pour ce faire, nous devons d'abord identifier les indices de canaux pertinents. En se référant au tracé 2D des capteurs mentionné précédemment, nous pourrions sélectionner les canaux suivants pour les ROIs gauche et droit, respectivement :

```
left = ["eeg17", "eeg18", "eeg25", "eeg26"]  
right = ["eeg23", "eeg24", "eeg34", "eeg35"]  
  
left_ix = mne.pick_channels(l_aud.info["ch_names"], include=left)  
right_ix = mne.pick_channels(l_aud.info["ch_names"], include=right)
```

Maintenant, nous pouvons créer un nouvel objet Evoked avec deux canaux virtuels (un pour chaque ROI).

```
roi_dict = dict(left_ROI=left_ix, right_ROI=right_ix)
roi_evoked = mne.channels.combine_channels(l_aud, roi_dict,
method="mean")
print(roi_evoked.info["ch_names"])
roi_evoked.plot()
```

Applying baseline correction (mode: mean)
['left_ROI', 'right_ROI']

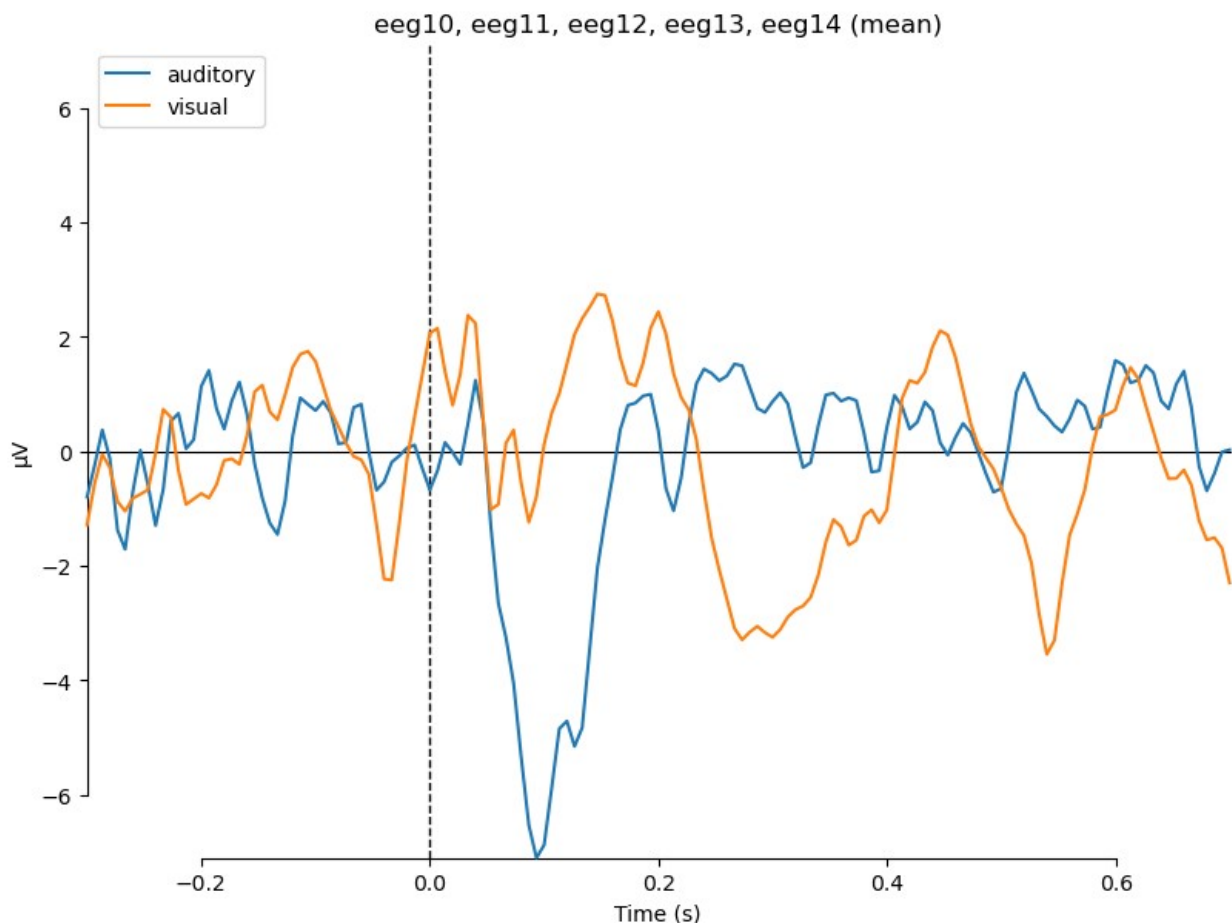


Comparaison des conditions

Si nous voulons comparer les stimuli auditifs aux stimuli visuels, la fonction `mne.viz.plot_compare_evokeds()` est très utile. Par défaut, cette fonction combine tous les canaux de chaque objet `evoked` en utilisant le GFP (ou le RMS pour les canaux MEG). Ici, nous spécifions plutôt de combiner par moyenne, et de restreindre cette combinaison à un sous-ensemble de canaux en passant l'argument `picks`.

```
evokeds = dict(auditory=l_aud, visual=l_vis)
picks = [f"eeg{n}" for n in range(10, 15)]
mne.viz.plot_compare_evokeds(evokeds, picks=picks, combine="mean")
```

combining channels using "mean"
combining channels using "mean"



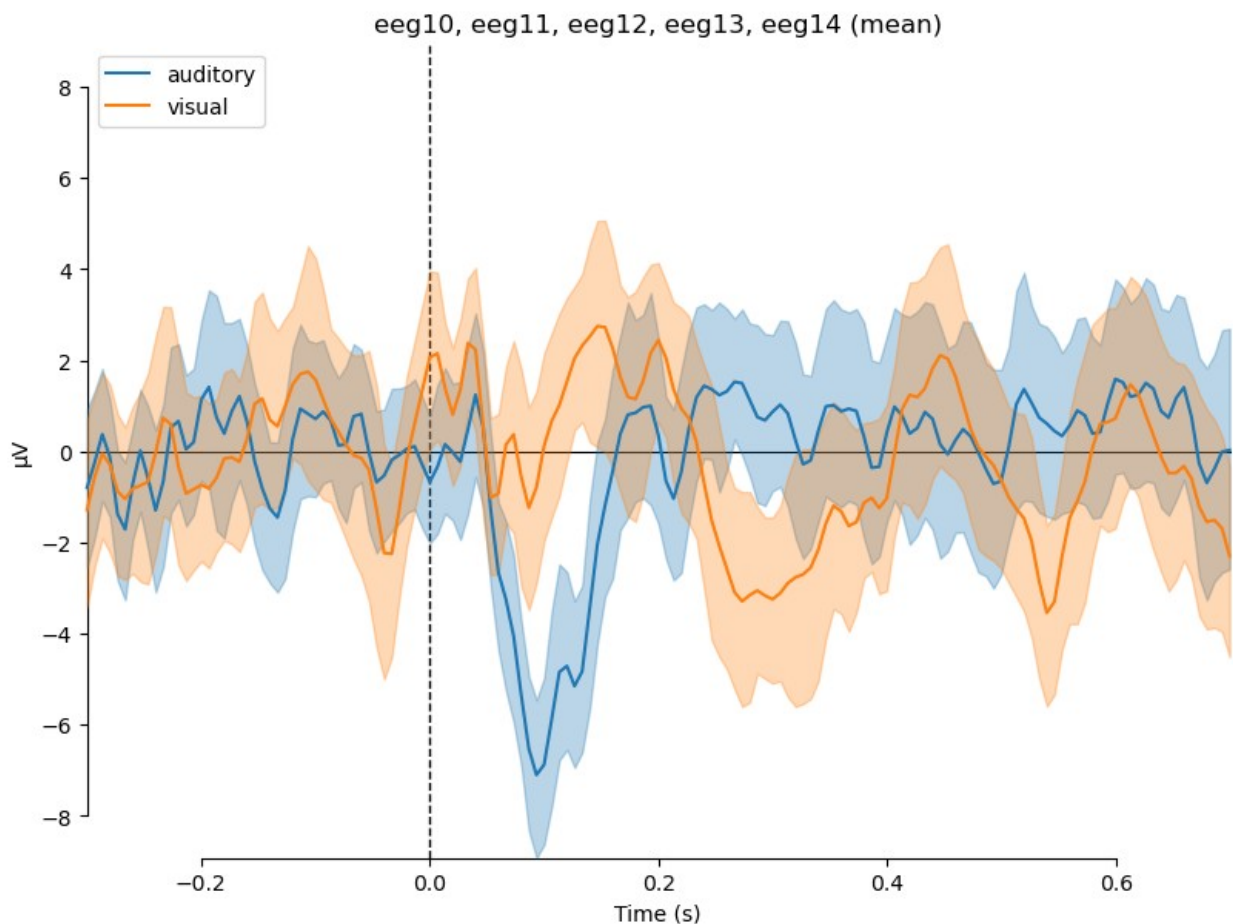
[<Figure size 800x600 with 1 Axes>]

Nous pouvons également générer des intervalles de confiance en traitant chaque époque comme une observation distincte, en utilisant `iter_evoked()`. Un intervalle de confiance entre les

subjects could also be obtained by passing a list of Evoked objects (one per subject) to the function `plot_compare_evoked()`.

```
evoked = dict(
    auditory=list(epochs["auditory/left"].iter_evoked()),
    visual=list(epochs["visual/left"].iter_evoked()),
)
mne.viz.plot_compare_evoked(evoked, combine="mean", picks=picks)

combining channels using "mean"
combining channels using "mean"
```

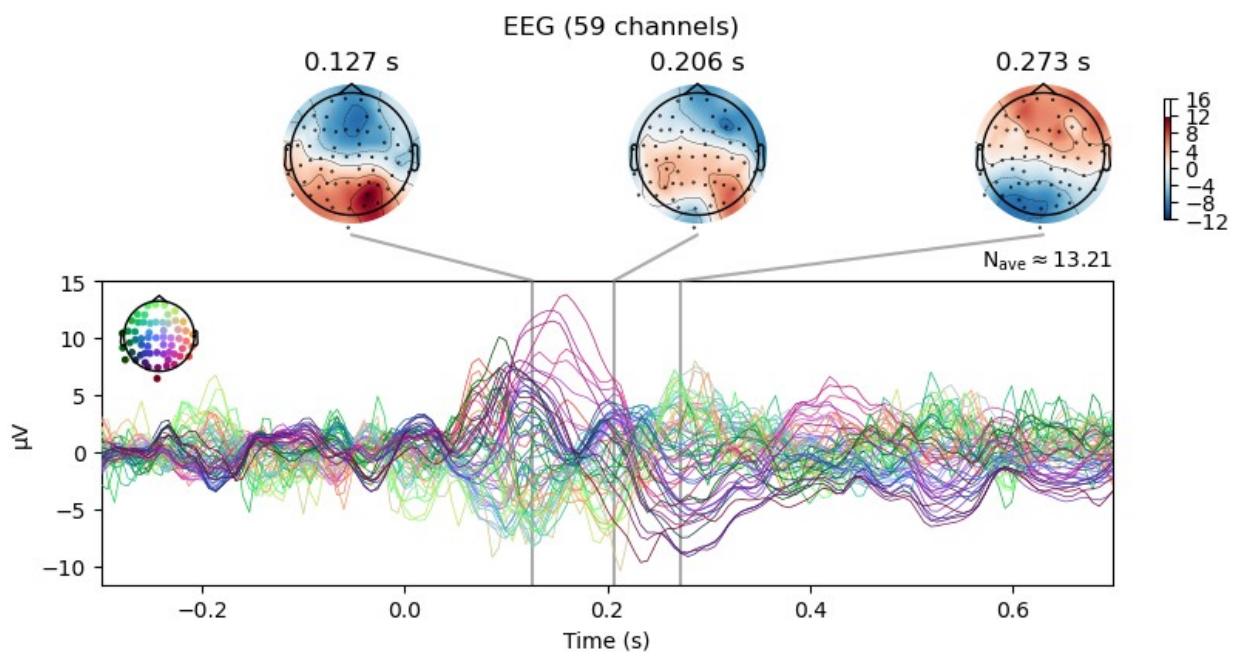
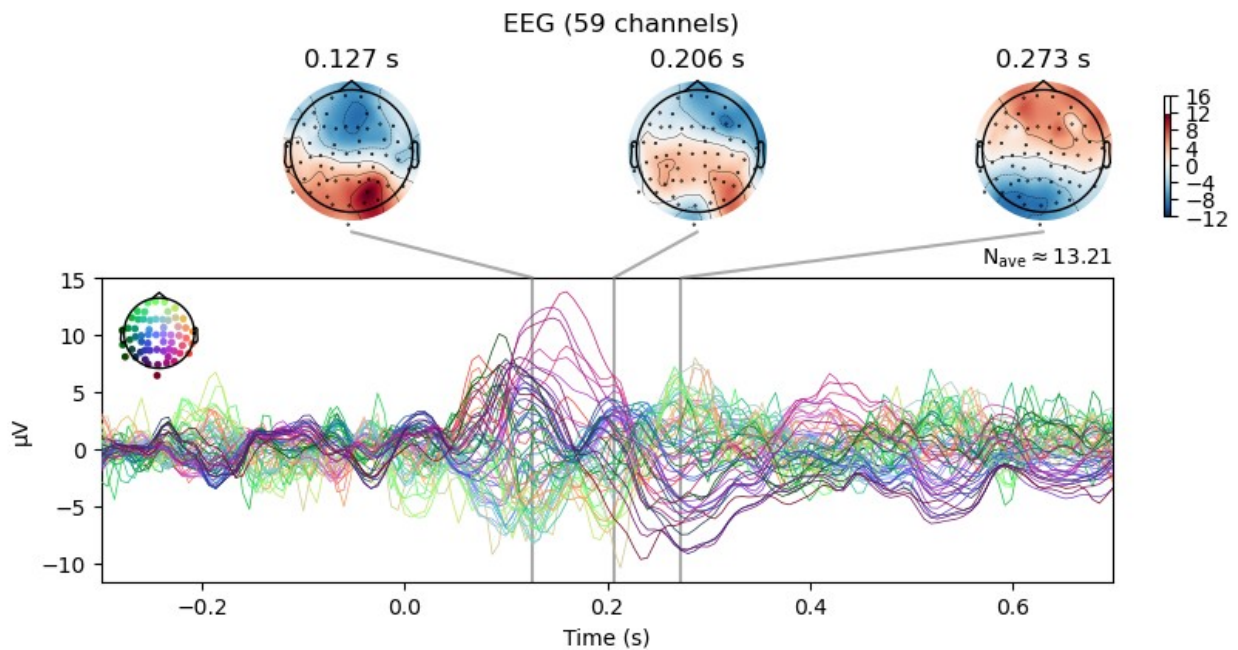


[<Figure size 800x600 with 1 Axes>]

We can also compare conditions by subtracting one Evoked object from another, using the function `mne.combine_evoked()` (this function also takes care of grouping epochs without subtraction).

```
aud_minus_vis = mne.combine_evoked([l_aud, l_vis], weights=[1, -1])
aud_minus_vis.plot_joint()
```

Projections have already been applied. Setting proj attribute to True.



```
grand_average = mne.grand_average([l_aud, l_vis])  
print(grand_average)
```

Setting channel interpolation method to {'eeg': 'spline'}.
Interpolating bad channels.

Automatic origin fit: head of radius 91.0 mm
Computing interpolation matrix from 59 sensor positions

```
Interpolating 1 sensors
Setting channel interpolation method to {'eeg': 'spline'}.
Interpolating bad channels.
    Automatic origin fit: head of radius 91.0 mm
Computing interpolation matrix from 59 sensor positions
Interpolating 1 sensors
Identifying common channels ...
<Evoked | 'Grand average (n = 2)' (average, N=2), -0.29969 – 0.69928
s, baseline -0.299693 – 0 s, 60 ch, ~3.0 MiB>
```

Pour combiner les conditions, il est également possible d'utiliser les tags HED dans les noms de condition lors de la sélection des époques à moyenner. Par exemple, nous avons les noms de condition suivants :

```
list(event_dict)

['auditory/left',
 'auditory/right',
 'visual/left',
 'visual/right',
 'face',
 'buttonpress']
```

Nous pouvons sélectionner les conditions auditives (gauche et droite ensemble) en passant :

```
epochs["auditory"].average()

<Evoked | '0.48 × auditory/left + 0.52 × auditory/right' (average,
N=52), -0.29969 – 0.69928 s, baseline -0.299693 – 0 s, 60 ch, ~3.0
MiB>
```

Mesures d'amplitude et de latence

Dans la recherche sur les ERP, il est courant d'extraire des mesures d'amplitude ou de latence pour les comparer entre différentes conditions. De nombreuses mesures peuvent être extraites des ERP, et plusieurs d'entre elles sont détaillées (ainsi que leurs forces et faiblesses respectives) dans le chapitre 9 de Luck.

Cette partie du tutoriel démontrera comment extraire trois mesures courantes :

Latence de pic

Amplitude de pic

Amplitude moyenne

Latence et amplitude de pic

Les mesures d'amplitude et de latence les plus courantes sont les mesures de pic. Les mesures de pic correspondent essentiellement à l'amplitude maximale du signal dans une fenêtre temporelle spécifiée et au point temporel (ou latence) auquel cette amplitude de pic s'est produite.

Les mesures de pic peuvent être obtenues en utilisant la méthode `get_peak()`. Il y a deux points importants à souligner concernant `get_peak()`. Premièrement, elle trouve le pic le plus fort en examinant tous les canaux du type sélectionné qui sont disponibles dans l'objet `Evoked`. Par conséquent, si vous souhaitez restreindre la recherche à un groupe de canaux ou à un seul canal, vous devez d'abord utiliser les méthodes `pick()` ou `pick_channels()`. Deuxièmement, la méthode `get_peak()` peut trouver différents types de pics en utilisant l'argument `mode`. Il existe trois options :

`mode='pos'`: trouve le pic avec une tension positive (ignore les tensions négatives)

`mode='neg'`: trouve le pic avec une tension négative (ignore les tensions positives)

`mode='abs'`: trouve le pic avec la plus grande tension absolue, quel que soit le signe (positif ou négatif)

L'exemple suivant montre comment trouver le premier pic positif dans l'ERP (c'est-à-dire le P100) pour la condition visuelle gauche (c'est-à-dire l'objet `l_vis Evoked`). La fenêtre temporelle utilisée pour rechercher le pic s'étend de 0,08 à 0,12 s. Cette fenêtre a été sélectionnée parce que c'est le moment où le P100 se produit typiquement. Notez que tous les canaux 'eeg' sont soumis à la méthode `get_peak()`.

```
# Define a function to print out the channel (ch) containing the
# peak latency (lat; in msec) and amplitude (amp, in  $\mu$ V), with the
# time range (tmin and tmax) that was searched.
# This function will be used throughout the remainder of the tutorial.
def print_peak_measures(ch, tmin, tmax, lat, amp):
    print(f"Channel: {ch}")
    print(f"Time Window: {tmin * 1e3:.3f} - {tmax * 1e3:.3f} ms")
    print(f"Peak Latency: {lat * 1e3:.3f} ms")
    print(f"Peak Amplitude: {amp * 1e6:.3f}  $\mu$ V")

# Get peak amplitude and latency from a good time window that contains
# the peak
good_tmin, good_tmax = 0.08, 0.12
ch, lat, amp = l_vis.get_peak(
    ch_type="eeg", tmin=good_tmin, tmax=good_tmax, mode="pos",
    return_amplitude=True
)

# Print output from the good time window that contains the peak
print("** PEAK MEASURES FROM A GOOD TIME WINDOW **")
print_peak_measures(ch, good_tmin, good_tmax, lat, amp)
```

```
** PEAK MEASURES FROM A GOOD TIME WINDOW **
```

```
Channel: eeg55
```

```
Time Window: 80.000 - 120.000 ms
```

```
Peak Latency: 86.578 ms
```

```
Peak Amplitude: 6.508  $\mu$ V
```

Le résultat montre que le canal eeg55 a eu le pic positif maximal dans la fenêtre temporelle choisie parmi tous les canaux 'eeg' recherchés. En pratique, on pourrait vouloir extraire le pic pour une région d'intérêt a priori ou un seul canal selon l'étude. Cela peut être fait en combinant les méthodes `pick()` ou `pick_channels()` avec la méthode `get_peak()`.

Ici, supposons que nous pensions que les effets d'intérêt se produiront au niveau d'eeg59.

```
# First, return a copy of l_vis to select the channel from
```

```
l_vis_roi = l_vis.copy().pick("eeg59")
```

```
# Get the peak and latency measure from the selected channel
```

```
ch_roi, lat_roi, amp_roi = l_vis_roi.get_peak(  
    tmin=good_tmin, tmax=good_tmax, mode="pos", return_amplitude=True  
)
```

```
# Print output
```

```
print("** PEAK MEASURES FOR ONE CHANNEL FROM A GOOD TIME WINDOW **")
```

```
print_peak_measures(ch_roi, good_tmin, good_tmax, lat_roi, amp_roi)
```

```
** PEAK MEASURES FOR ONE CHANNEL FROM A GOOD TIME WINDOW **
```

```
Channel: eeg59
```

```
Time Window: 80.000 - 120.000 ms
```

```
Peak Latency: 86.578 ms
```

```
Peak Amplitude: 5.713  $\mu$ V
```

Alors que les latences de pic sont identiques dans les canaux eeg55 et eeg59, les amplitudes de pic diffèrent. Cette approche peut également être appliquée aux canaux virtuels créés avec la fonction `combine_channels()` et aux ondes de différence créées avec la fonction `mne.combine_evoked()` (voir `aud_minus_vis` dans la section "Comparer les conditions" ci-dessus).

Les mesures de pic sont très sensibles au bruit de haute fréquence dans le signal (pour plus de détails, voir). Plus précisément, le bruit de haute fréquence biaise positivement les mesures d'amplitude de pic. Ce biais peut fausser les comparaisons entre des conditions où les ERP diffèrent en termes de niveau de bruit de haute fréquence, par exemple lorsque les conditions diffèrent par le nombre d'essais contribuant à l'ERP. Une façon d'éviter cela est d'appliquer un filtre passe-bas non causal à l'ERP. Les filtres passe-bas réduisent la contribution du bruit de haute fréquence en lissant les fluctuations rapides (c'est-à-dire à haute fréquence) du signal (voir "Informations générales sur le filtrage"). Bien que cela puisse réduire le biais positif dans les mesures d'amplitude de pic causé par le bruit de haute fréquence, le filtrage passe-bas de l'ERP peut introduire des défis dans l'interprétation des mesures de latence de pic pour les effets d'intérêt.

Si vous utilisez des mesures de pic, il est essentiel d'inspecter visuellement les données pour vous assurer que la fenêtre temporelle sélectionnée contient réellement un pic. La méthode `get_peak` détecte la tension maximale ou minimale dans la plage de temps spécifiée et renvoie la latence et l'amplitude de ce pic. Il n'y a aucune garantie que cette méthode renverra un pic réel. Au lieu de cela, elle peut renvoyer une valeur sur le front montant ou descendant d'un pic que nous essayons de trouver.

L'exemple suivant démontre pourquoi l'inspection visuelle est cruciale. Ci-dessous, nous utilisons une fenêtre temporelle délibérément mauvaise (0,095 à 0,135 s) pour rechercher un pic dans le canal eeg59.

```
# Get BAD peak measures
bad_tmin, bad_tmax = 0.095, 0.135
ch_roi, bad_lat_roi, bad_amp_roi = l_vis_roi.get_peak(
    mode="pos", tmin=bad_tmin, tmax=bad_tmax, return_amplitude=True
)

# Print output
print("** PEAK MEASURES FOR ONE CHANNEL FROM A BAD TIME WINDOW **")
print_peak_measures(ch_roi, bad_tmin, bad_tmax, bad_lat_roi,
    bad_amp_roi)

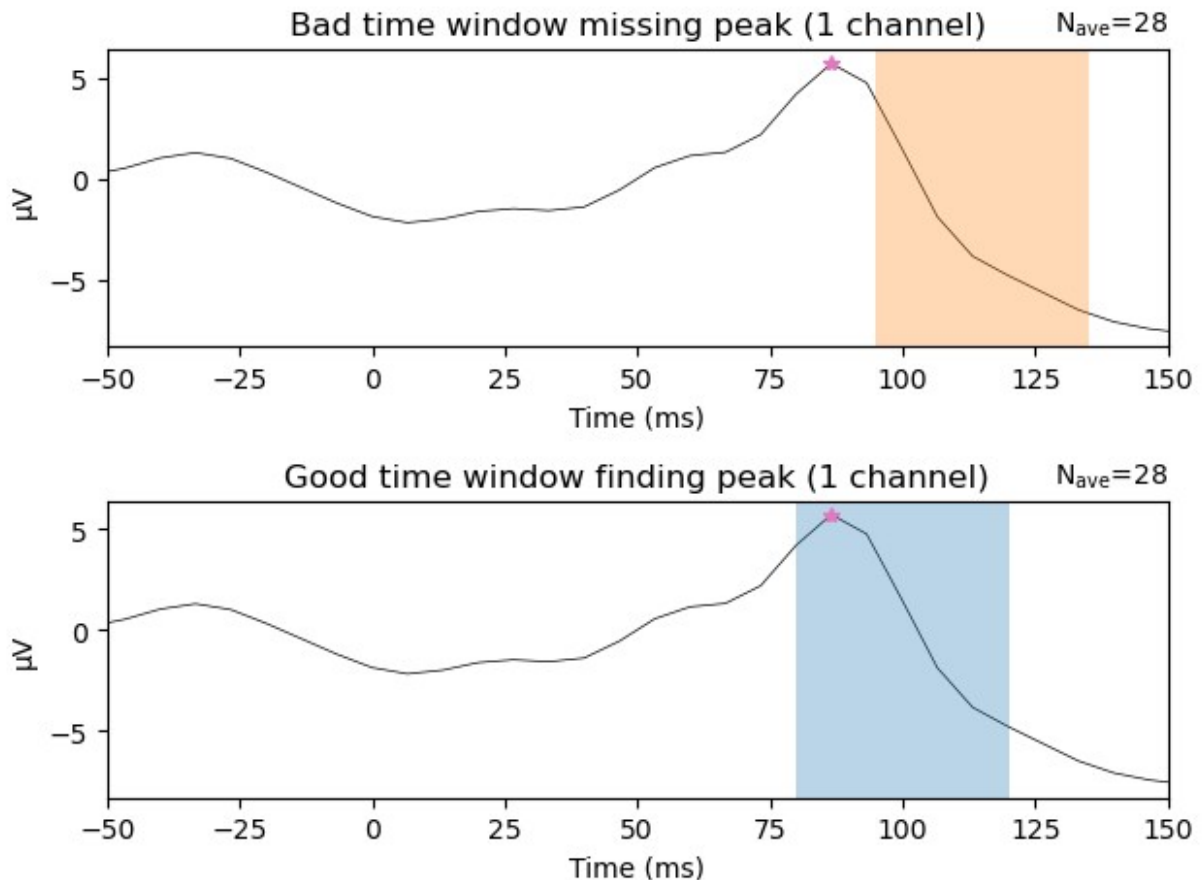
** PEAK MEASURES FOR ONE CHANNEL FROM A BAD TIME WINDOW **
Channel: eeg59
Time Window: 95.000 - 135.000 ms
Peak Latency: 99.898 ms
Peak Amplitude: 1.487  $\mu$ V
```

S'il n'y avait que les valeurs ci-dessus, il serait difficile de savoir si elles identifient réellement un pic dans l'ERP. En fait, la fenêtre temporelle de 0,095 à 0,135 s ne contient pas le véritable pic, comme le montre le panneau supérieur ci-dessous. La mauvaise fenêtre temporelle (surlignée en orange) ne contient pas le vrai pic (l'étoile rose). En revanche, la fenêtre temporelle définie initialement (0,08 à 0,12 s ; surlignée en bleu) renvoie un pic réel au lieu d'un simple maximum ou minimum dans la fenêtre temporelle recherchée. L'inspection visuelle vous aidera toujours à vous assurer que les valeurs retournées sont des pics réels.

```
fig, axs = plt.subplots(nrows=2, ncols=1, layout="tight")
words = (("Bad", "missing"), ("Good", "finding"))
times = (np.array([bad_tmin, bad_tmax]), np.array([good_tmin,
    good_tmax]))
colors = ("C1", "C0")

for ix, ax in enumerate(axs):
    title = "{} time window {} peak".format(*words[ix])
    l_vis_roi.plot(axes=ax, time_unit="ms", show=False, titles=title)
    ax.plot(lat_roi * 1e3, amp_roi * 1e6, marker="*", color="C6")
    ax.axvspan(*(times[ix] * 1e3), facecolor=colors[ix], alpha=0.3)
    ax.set_xlim(-50, 150) # Show zoomed in around peak
```

Need more than one channel to make topography for eeg. Disabling interactivity.
Need more than one channel to make topography for eeg. Disabling interactivity.



Amplitude moyenne

Une pratique courante dans les études ERP consiste à définir une composante (ou un effet) comme l'amplitude moyenne dans une fenêtre temporelle spécifiée. L'un des avantages de cette approche est qu'elle est moins sensible au bruit de haute fréquence (par rapport aux mesures d'amplitude de crête), car la moyenne sur une fenêtre temporelle agit comme un filtre passe-bas (voir la discussion dans la section précédente sur la latence et l'amplitude de crête).

Lorsque l'on utilise des mesures d'amplitude moyenne, sélectionner la fenêtre temporelle en se basant sur l'effet d'intérêt (par exemple, la différence entre deux conditions) peut augmenter la probabilité de trouver des faux positifs dans vos résultats. Il existe d'autres, et meilleures, façons d'identifier une fenêtre temporelle à utiliser pour extraire les mesures d'amplitude moyenne. Premièrement, vous pouvez utiliser une fenêtre temporelle a priori basée sur des recherches antérieures. Une deuxième option est de définir une fenêtre temporelle à partir d'une condition ou d'un ensemble d'essais indépendants non utilisés dans l'analyse (par exemple, un

"localisateur"). Une troisième approche consiste à définir une fenêtre temporelle en utilisant la moyenne générale entre les conditions. Cette dernière approche n'est pas circulaire car la moyenne entre les conditions et la différence de condition sont indépendantes l'une de l'autre. Les problèmes discutés ci-dessus s'appliquent également à la sélection des canaux utilisés pour l'analyse.

L'exemple suivant démontre comment extraire l'amplitude moyenne de la condition visuelle gauche (c'est-à-dire l'objet `l_vis Evoked`) à partir de canaux et de fenêtres temporelles sélectionnés. La stimulation du champ visuel gauche augmente l'activité neurale du cortex visuel dans l'hémisphère contralatéral (c'est-à-dire droit). Nous pouvons tester cela en examinant l'amplitude de l'ERP pour la stimulation du champ visuel gauche sur les canaux droits (contralatéraux) et gauches (ipsilatéraux). Les canaux utilisés pour cette analyse sont eeg54 et eeg57 (hémisphère gauche), et eeg59 et eeg55 (hémisphère droit). La fenêtre temporelle utilisée est de 0,08 (`good_tmin`) à 0,12 s (`good_tmax`), car cela correspond au moment où la P100 se produit généralement. La P100 est sensible à la stimulation du champ visuel gauche et droit. L'amplitude moyenne est extraite des quatre canaux ci-dessus et stockée dans un `pandas.DataFrame`.

```
# Select all of the channels and crop to the time window
channels = ["eeg54", "eeg57", "eeg55", "eeg59"]
hemisphere = ["left", "left", "right", "right"]
l_vis_mean_roi = l_vis.copy().pick(channels).crop(tmin=good_tmin,
tmax=good_tmax)

# Extract mean amplitude in  $\mu V$  over time
mean_amp_roi = l_vis_mean_roi.data.mean(axis=1) * 1e6

# Store the data in a data frame
mean_amp_roi_df = pd.DataFrame(
    {
        "ch_name": l_vis_mean_roi.ch_names,
        "hemisphere": ["left", "left", "right", "right"],
        "mean_amp": mean_amp_roi,
    }
)

# Print the data frame
print(mean_amp_roi_df.groupby("hemisphere").mean(numeric_only=True))
```

	mean_amp
hemisphere	
left	-0.279764
right	0.685840

Comme démontré dans cet exemple, l'amplitude moyenne était plus élevée et positive dans les canaux de l'hémisphère droit par rapport aux canaux de l'hémisphère gauche. Il convient de réaffirmer que les fenêtres spatiales et temporelles utilisées dans l'analyse doivent être déterminées de manière indépendante (par exemple, définies a priori à partir de recherches antérieures, d'un "localisateur" ou d'une autre condition indépendante) et non basées sur les données que vous utiliserez pour tester vos hypothèses.

L'exemple peut être modifié pour extraire l'amplitude moyenne de tous les canaux et stocker le résultat dans un pandas.DataFrame. Cela peut être utile pour les analyses statistiques menées dans d'autres langages de programmation.

```
# Extract mean amplitude for all channels in l_vis (including `eog`)
l_vis_cropped = l_vis.copy().crop(tmin=good_tmin, tmax=good_tmax)
mean_amp_all = l_vis_cropped.data.mean(axis=1) * 1e6
mean_amp_all_df = pd.DataFrame(
    {"ch_name": l_vis_cropped.info["ch_names"], "mean_amp":
mean_amp_all}
)
mean_amp_all_df["tmin"] = good_tmin
mean_amp_all_df["tmax"] = good_tmax
mean_amp_all_df["condition"] = "Left/Visual"
with pd.option_context("display.max_columns", None):
    print(mean_amp_all_df.head())
    print(mean_amp_all_df.tail())
```

	ch_name	mean_amp	tmin	tmax	condition
0	eeg01	-1.689513	0.08	0.12	Left/Visual
1	eeg02	-0.078492	0.08	0.12	Left/Visual
2	eeg03	-0.381191	0.08	0.12	Left/Visual
3	eeg04	-1.609264	0.08	0.12	Left/Visual
4	eeg05	-1.113717	0.08	0.12	Left/Visual
	ch_name	mean_amp	tmin	tmax	condition
55	eeg56	2.064174	0.08	0.12	Left/Visual
56	eeg57	-0.259681	0.08	0.12	Left/Visual
57	eeg58	0.156472	0.08	0.12	Left/Visual
58	eeg59	0.802306	0.08	0.12	Left/Visual
59	eeg60	0.878811	0.08	0.12	Left/Visual